Forensic Architectural Audit: Reconstructing the Lost Governance of DP-OPS-0039 and the Hardened TASK.md Standard

1. Executive Summary: The Architecture of Loss and Recovery

The remediation of the Stela repository's active work surface—specifically the TASK.md file—requires a forensic reconstruction of the governance standards established in **DP-OPS-0039** and subsequently hardened in **DP-OPS-0044**. The user's grievance regarding missing sections and the failure of the reconstructed file to solve "DP issues" is not merely a complaint about absent text; it is a precise identification of **System Drift** and **Canon Divergence**.[1] The Stela architecture is designed to be self-correcting through the rigid enforcement of "Physical Laws" defined in the **Policy of Truth (PoT.md)**.[1] When these laws are violated, or when the "Active Work Surface" (TASK.md) fails to reflect the rigorous standards of the "History Ledger" (SoP.md), the system enters a catastrophic failure state known as **Canon Drift**.[1]

This report serves as a comprehensive forensic audit and architectural blueprint. It deconstructs the repository's constitutional framework to isolate exactly what was lost from the **DP-OPS-0039** mandate. The analysis confirms that the "lost" components are not random deletions but specific, high-value governance mechanisms: **Explicit Closeout Pointers**, **Quantitative Reporting Requirements**, **Scope Boundary Permissions**, and the integration of **Hot Zone Forensics (S-LEARN-06)**.[1] Furthermore, the report establishes the definitive standard for **Formally Robust Results**, a concept critical to the repository's "Pointer-First" logic, which demands that every action be traceable, verifiable, and immutable.[1]

By synthesizing the "Hybrid-Hardened" structural requirements—designed to satisfy the divergent cognitive architectures of document-centric and data-centric AI models—this document provides the roadmap to restore TASK.md to its intended state: a deterministic, enforcement-grade contract that governs the intersection of human intent and machine execution.[1]

---

2. Constitutional Framework: The Physics of Stela

To understand what is missing from the active work surface, one must first understand the "physical laws" that govern the repository. In the Stela system, governance is not a set of guidelines; it is a set of inviolable constraints enforced by automation. The "Policy of Truth" (PoT.md) acts as the constitution, establishing the axioms that define existence within the repository.[1]

2.1 The Hierarchy of Truth

The integrity of the repository relies on a strict **Source of Truth (SSOT) Read-in Order**. This hierarchy ensures that no operational activity can contradict the foundational laws of the system. The breakdown of this hierarchy reveals why a malformed TASK.md is a critical failure. The system demands that any intelligent agent (human or AI) ingest these documents in a specific sequence to build a valid "worldview" before attempting any action.[1]

1. PoT.md (The Constitution): This is the final authority on staffing, jurisdiction, and enforcement.[1] It defines the "Physical Laws" of the environment. If a conflict exists between the PoT and any other file, the PoT prevails. It establishes the "Filing Doctrine" and the "Axioms" that prevent chaos. It is the bedrock upon which all other logic rests. Without it, the system has no definition of "success" or "failure."

2. SoP.md (The History): The State of Play serves as the history ledger and shipment record.[1] It provides the context of *why* the system is in its current state. Crucially, it is a record of *what happened*, not a rulebook for *what must happen*, yet it validates the legitimacy of the active contract. It acts as the "memory" of the system, preventing circular work or the re-introduction of previously resolved bugs.

3. TASK.md (The Contract): This is the active work surface.[1] It contains the current **Dispatch Packet (DP)**. It is the bridge between the static laws of the PoT and the kinetic action of the work. If this file is incomplete, the "Contract" is void, and the worker (AI or Human) is operating without a valid license. It is the only place where the abstract laws of the repository interact with the concrete reality of code changes.[1]

4. docs/MAP.md (The Terrain): This provides continuity wayfinding, ensuring that agents can navigate the repository structure without hallucinating non-existent paths.[1] It serves as the spatial reference, preventing agents from getting lost in the directory structure or misplacing files.

2.2 The Filing Doctrine and Jurisdictional Separation

A robust TASK.md must respect the **Filing Doctrine**, which separates the repository into four distinct jurisdictions to prevent "Context Hazards".[1] This separation is not merely organizational; it is a security feature designed to prevent "Context Contamination."

- **ops/ (Run):** This is the "Kinetic Layer." It contains executable logic, binaries, manifests,

and automation. Narrative explanations are forbidden here; this is the domain of pure action. The ops/bin/open and ops/bin/dump binaries reside here, serving as the tools that manipulate the TASK.md state. Any attempt by an agent to write narrative text into this directory is a violation of the Filing Doctrine.[1]

- **docs/ (Explain):** This is the "Semantic Layer." It contains manuals, rationale, and the libraries of Agents, Skills, and Tasks. A critical axiom of the PoT is the **Context Hazard**: including docs/library/ files in the global context manifest is a system failure.[1] This necessitates that TASK.md utilize **pointers** to these files rather than embedding them, a nuance often lost in reconstruction.
- **projects/ (Work):** This is the "Output Layer," housing the payload code. It is sandboxed to prevent agents from rewriting the operating system (ops/) while attempting to perform a task. It represents the "product" of the system, separated from the "machinery" that produces it.[1]
- **storage/ (Trash):** This layer is designated for local artifacts and handoffs. It is explicitly defined as "never canon," ensuring that the ephemeral data generated during a task does not pollute the repository's permanent history. This is where the RESULTS.md and OPEN artifacts live before they are processed.[1]

2.3 The "Dirty State" Paradox and the TASK.md Exception

The PoT establishes a "Dirty State" as a system failure triggered by uncommitted changes in the main branch. This rule forces a "STOP WORK" condition to prevent divergent histories. However, TASK.md is granted a unique **Constitutional Exemption**.[1] It is the *only* file permitted to remain "dirty" (uncommitted) during active operations. This exemption allows it to function as a real-time dashboard for logging progress and session state.

The "lost" aspect of many reconstructed TASK.md files is the structure that supports this exemption. A proper TASK.md must contain mutable sections (like a **Work Log**) that are designed to be written to by the agent, while the rest of the file (the **DP Contract**) remains immutable. If the file is wholly static or wholly mutable, it violates this delicate constitutional balance.[1]

2.4 The Mechanics of Enforcement

The repository employs a sophisticated system of "Policing" to ensure that TASK.md remains compliant. This is not left to human vigilance; it is enforced by code.

- **Task Policing (task-policing.yml):** This workflow enforces a specific rule: if any "gate surface" is modified, TASK.md *must* be updated. Gate surfaces include ops/bin/open, ops/bin/dump, tools/lint/dp.sh, and docs/MANUAL.md. If a user modifies the manual but fails to update the active task contract, the system rejects the change.[1]
- **SoP Policing (sop_policing.yml):** This workflow ensures that the history ledger is kept in sync with the canon. If TASK.md (a canon surface) changes, SoP.md must also change. This "double-entry bookkeeping" ensures that every change to the work surface is

recorded in the history logs.[1]

---

3. Forensic Reconstruction of DP-OPS-0039

The user's query explicitly references **DP-OPS-0039** as the standard that "promised to solve all my DP issues." By analyzing the SoP.md ledger entry for this Dispatch Packet, we can reconstruct the exact requirements that were likely lost in the user's current version. This packet was not a minor update; it was a fundamental refactoring of the **Task System** to enforce quantitative rigor and pointer-first logic.[1]

3.1 The "Lost" Doctrine: Explicit Closeout Pointers

The ledger entry for **DP-OPS-0039** (dated 2026-02-10 01:27:17 UTC) explicitly states: *"Updated task doctrine to require explicit Closeout pointers in Execution Logic"*.[1]

What was lost: In standard or "default" TASK.md templates, the Execution Logic often ends with a generic "Verify work" or "Submit PR" step. DP-39 mandated a **Pointer-First** approach. The final step of *every* task must not be a generic instruction but a hard pointer to **TASK.md Section 4**.

The Reconstructed Requirement:

- **Old/Bad Pattern:** "Step 5: Verify results and create a PR."
- **DP-39 Hardened Pattern:** "Step 5: Complete Closeout per TASK.md Section 4."

This pointer is critical because Section 4 contains the mandatory **Closing Block**, which requires specific, distinct entries for the Commit Header, PR Title, and Squash Stub. Without this pointer, agents drift into "hallucinated compliance," inventing their own closeout procedures that fail to update the SoP.md ledger, thus violating the **Audit Readiness** axiom.[1]

3.2 Quantitative Rigor in Results

The second major component of DP-39 was the requirement for *"quantitative reporting capture in RESULTS"*.[1] The **Behavioral Logic Standard** in PoT.md enforces "Linguistic Precision," which specifically mandates "Quantitative reporting required for deviations from protocol".[1]

What was lost: Many reconstructed task files revert to "vibes-based" reporting (e.g., "I fixed the bug," "Tests passed"). This is unacceptable under the Stela governance model.

The DP-39 Standard: The results must be objective and measurable.

- *Bad:* "Updated the linting script."
- *Robust:* "Updated tools/lint/task.sh. git diff --stat shows +14 insertions, -2 deletions. Ran verification: bash tools/verify.sh passed (Exit Code 0)."

The "lost" section in the user's TASK.md is likely the specific **Receipt Template** or **Results Block** that forces the agent to input these metrics. Without a structural slot for "Raw Outputs" and "Diff Stats," agents will revert to conversational summaries.[1]

3.3 Scope Boundary and Closeout Allowances

DP-39 involved refactoring **B-TASK-01 through B-TASK-06** to include *"Closeout scope allowances"*.[1]

What was lost: A common friction point for AI agents is the **Scope Boundary**. Agents are often given strict instructions: "Do not edit files outside of projects/." However, the **Closeout** routine requires them to edit SoP.md (to log the shipment) and TASK.md (to update the work log). If the TASK.md Scope Boundary does not explicitly *allow* these actions, the agent hits a "Logical Deadlock"—it is ordered to close out, but forbidden from touching the files necessary to do so.[1]

The Reconstructed Requirement:

The TASK.md must include a **Scope Boundary** section that explicitly whitelists the governance surfaces required for the closeout routine:

- **Allowed:** "Edit SoP.md to append the shipment record."
- **Allowed:** "Edit TASK.md to update the Work Log and set status to COMPLETE."

3.4 Hot Zone Forensics (S-LEARN-06)

DP-39 added **S-LEARN-06 Hot Zone Forensics guidance**.[1] This is a "Just-in-Time" (JIT) skill that agents must load when working in critical areas of the repository.

What was lost: The reconstructed TASK.md likely fails to reference this skill in its **Required Context Load** or **JIT Skills** section. Without this pointer, agents working on high-risk files (like auth logic or payment gateways) will operate without the necessary forensic constraints, potentially introducing security regressions. The inclusion of this skill is a mandatory check for any task that touches the "Hot Zone".[1]

---

4. The Anatomy of the Hardened TASK.md

Based on the forensic audit of **DP-OPS-0044** (which hardened the contract further) and the requirements of **DP-OPS-0039**, the "Hardened TASK.md" is not a simple checklist. It is a "Hybrid-Hardened" structure designed to satisfy the cognitive architectures of both **Document-Centric** models (like ChatGPT) and **Data-Centric** models (like Gemini).[1]

The user's "reconstructed" file is likely missing the specific **structural hardening** elements that prevent "Instruction Skipping" and "Attention Dilution." These elements are not

decorative; they are functional constraints that force the AI model to behave deterministically.

4.1 The Hybrid-Hardened Schema

To be "formally robust," the TASK.md must utilize a **Strict Decimal Schema** (e.g., 3.1, 3.2) for its headers.[1]

- **Why it matters:** ChatGPT relies on visual hierarchy. A Level-2 header (##) signals a major topic change, while a Level-3 header (###) signals a sub-task. If the decimal schema is broken or missing (e.g., just using bullet points), the model loses track of "Scope" and may skip critical verification steps.
- **Data-Centric Requirement:** Gemini treats the document as a dataset. The decimal headers act as "Keys" for its internal parsing. Consistent naming (e.g., "3.1 Freshness Gate") allows it to reliably locate constraints.

Table 1: Cognitive Alignment of Hardened Schema

Component	ChatGPT Requirement (Document-Centric)	Gemini Requirement (Data-Centric)	Implementation in Hardened TASK.md
Headers	Visual Hierarchy (## vs ###)	Consistent Keys	Strict Decimal Schema (3.1, 3.2)
Fields	Bulleted Lists	Key-Value Pairs	Unified Format: - **Key:** Value
Logic	Narrative Flow	Step-by-Step Instructions	Numbered Steps with Pointers
Constraints	Explicit Checklists	Allowed/Forbidden Keys	"Scope and Safety" Block

4.2 The Placeholder Paradox and Sanitized Tokens

One of the most critical "lost" aspects is likely the management of placeholders. Historical versions used loose tokens like [Insert Objective] or TBD. This created a vulnerability known as **"Template Debris"**.[1]

- **The Vulnerability:** When ChatGPT sees [Insert Objective], it often treats it as an imperative to *fill in the blank* using hallucinated or predicted text, rather than waiting for

the operator to provide the goal.
- **The Hardened Fix:** The current standard requires **Sanitized Tokens** like ENTER_VALUE. The logic dictates: "If ENTER_VALUE is present, STOP. Do not proceed." This transforms a potential hallucination vector into a hard **Stop Condition**.

4.3 Section 1: Session State (The Anchor)

This section is the "Consciousness" of the session. It must contain [1]:

- **Pointer:** Link to the specific OPEN artifact in storage/handoff/.
- **Active Branch:** Must match the OPEN artifact.
- **Base HEAD:** The Git hash at the start of the session.
- **Context Manifest:** A pointer to ops/lib/manifests/CONTEXT.md to verify the "worldview" of the agent.

Missing Logic: If this section is missing or generic, the agent suffers from "Identity Confusion," not knowing *when* or *where* it is operating. It forces the agent to verify its temporal location in the git history before taking action.

4.4 Section 2: Logic Pointers (The Law)

This section enforces the **Pointer-First** architecture. It must not *restate* the rules; it must *point* to them.[1]

- **Primary Constraint:** PoT.md (Policy of Truth).
- **Toolchain:** Pointers to tools/lint/style.sh, tools/verify.sh, etc.

Missing Logic: If the TASK.md tries to summarize the rules ("Don't use contractions") instead of pointing to PoT.md, it creates a "Stale Governance" risk. If the PoT changes, the Task file is outdated. The hardened version *must* use pointers.

4.5 Section 3: Current Dispatch Packet (DP)

This is the core of the user's grievance. This section must house the **DP Contract**.

- **3.1 Freshness Gate:** A mandatory check to ensure the agent is on the correct branch and commit. It must list the "Gate Artifacts" (OPEN, OPEN-PORCELAIN, Dump, Dump Manifest) with their specific hashes.[1]
- **3.2 Required Context Load:** A checklist of *exactly* what the agent needs to read. This must include negative constraints (e.g., "Do NOT load docs/library/agents") to prevent "Context Hazard" failures.[1]
- **3.3 Scope and Safety:** The "Context Fencing." It must explicitly forbid external data retrieval (Google Search) and restrict the agent to the provided dump.[1]
- **3.4 Execution Plan:** The step-by-step logic. As per DP-39, the final step *must* be a pointer to Section 4. It must use the "A-E Canon" (A, B, C, D, E) or decimal (3.4.1 - 3.4.5)

substructure to align with the linter.[1]

4.6 Section 4: Closeout (Mandatory)

This is likely the most damaged or missing section in the user's file. It is not just a "Done" checklist; it is a rigorous **Promotion Gate**.[1]

Required Subsection: 4.1 Thread Transition

- **Reset/Archive Rule:** Explicit instructions to append a **THREAD END** entry to the Work Log. This marker prevents "Semantic Debris"—ghost data from previous sessions—from leaking into the next task.[1]

4.7 Section 5: Work Log (Timestamped Continuity)

The **Work Log** is the only mutable part of the file. It requires a specific timestamped format to ensure continuity.[1]

- **Format:** YYYY-MM-DD HH:MM - THREAD START: DP-OPS-XXXX.
- **Next Action:** Every entry must conclude with a single NEXT: field to maintain the chain of custody for the agent's attention.

---

5. Formally Robust Results: The Definition of Done

The user requested "formally robust results." In the context of the Stela repository, this is a specific technical term defined by the **Forensic Provenance** standards.[1] A result is not "robust" because it is good; it is robust because it is **deterministic**, **verifiable**, and **machine-parsable**.

5.1 The "Receipt" Concept

The "Formally Robust Results" are generated as a **Receipt Bundle**. This is not a text description; it is a specific set of artifacts generated by the ops/bin/dump and ops/bin/context tools.[1]

The Receipt Must Include:

1. **RESULTS.md:** A file containing the raw output of verification scripts.
2. **OPEN-PORCELAIN:** A text file documenting the exact state of the working tree (porcelain status) at the start of the task.
3. **Dump Manifest:** A manifest listing every file included in the context, ensuring no hidden data influenced the decision.
4. **Git Diff Stat:** A quantitative measure of the change (e.g., +45, -12).

5.2 Metadata Keys and Provenance Blocks

Every artifact (code, agent, or skill) promoted to the repository must contain a **Provenance Block**.[1] If the user's TASK.md generates code without this block, it is non-compliant.

Required Metadata Keys:

- DP_ID: The Dispatch Packet ID (e.g., DP-OPS-0039).
- Created: Timestamp.
- Branch: The work branch.
- **HEAD**: The Git Hash at the moment of creation. *This is critical for forensic reconstruction.*
- Objective: The specific goal.
- Diff Stat: The quantitative impact.

5.3 Triple-Bookkeeping

Robust results require **Triple-Bookkeeping**.[1] A task is not "Done" until the change is reflected in three places:

1. **The File:** The actual code or documentation change.
2. **The Index:** Updating docs/INDEX.md or docs/library/INDEX.md.
3. **The Ledger:** Updating SoP.md (History) or docs/library/TASKS.md (Registry).

If the user's TASK.md does not explicitly force this triple update, it produces "Ghost Artifacts"—files that exist but are not recognized by the system's governance.

---

6. Policing and Enforcement: Why the "Lost" Sections Matter

The user's anger stems from the fact that without these "lost" sections, the repository's automated policing mechanisms will essentially "lock out" the user or fail every PR. The governance is enforced by **Gate Surfaces** defined in .github/workflows/task_policing.yml.[1]

6.1 The Gate Surface Trigger

The workflow enforces a rule: **"Enforce TASK update when gate surfaces change."**

If the user modifies ops/bin/open, ops/bin/dump, or tools/lint/dp.sh, the system checks if TASK.md was also modified. If TASK.md is missing the specific sections that reference these tools (e.g., the **Freshness Gate** calling ops/bin/open), the agent cannot logically update the file, and the PR fails.

6.2 The Context Hazard

The PoT.md defines a **Context Hazard** as the inclusion of library docs in the global context.[1] If the "lost" TASK.md structure doesn't include the **Negative Constraints** (Context Fencing) explicitly forbidding this, agents will lazily load the entire library. This triggers the tools/lint/context.sh linter, causing an immediate build failure.

---

7. Remediation Plan: Restoring the Lost Architecture

To "give back what was lost" and solve the user's DP issues, the TASK.md must be reconstructed with the following specific modifications, directly aligned with DP-OPS-0039 and DP-OPS-0044 standards.

7.1 Reinstating the Missing Sections

The reconstructed TASK.md must include:

1. **The Header Block:** Must use the **Strict Decimal Schema** (e.g., ## 3.1, ## 3.2) to satisfy the tools/lint/dp.sh validation logic.[1]
2. **The Freshness Gate (3.1):** Must explicitly list the OPEN, OPEN-PORCELAIN, Dump, and Dump Manifest artifacts with their specific hashes.
3. **The Allowlist (3.3):** Must be an explicit list of files. "Narrative" scopes are no longer permitted; the list must be machine-readable by the policing scripts.
4. **The Execution Plan (3.4):** Must follow the A-E Canon or 3.4.1-3.4.5 decimal structure.
5. **The Closeout Pointer:** The final step of the Execution Plan *must* read: "Complete Closeout per TASK.md Section 4."

7.2 The Results Template

The user needs the specific **RESULTS** skeleton. Based on the robust results standard, the file storage/handoff/DP-OPS-XXXX-RESULTS.md must adhere to this structure [1]:

DP-OPS-XXXX RESULTS

1. Verification Outputs (Raw)

[Capture raw stdout/stderr here]

- bash tools/lint/context.sh
- bash tools/lint/style.sh
- bash tools/lint/truth.sh
- bash tools/lint/dp.sh --test
- bash tools/lint/dp.sh TASK.md

- bash tools/verify.sh

2. Git Status

- Porcelain: [Output of git status --porcelain]
- Diff Stat: [Output of git diff --stat]
- Statement: "All changed files are within the DP-OPS-XXXX allowlist."

3. Gate Artifacts

- OPEN: storage/handoff/OPEN-work-branch-hash.txt
- Dump: storage/dumps/dump-platform-branch-hash.txt

4. Context Refresh

- Command:./ops/bin/llms --out-dir=.
- Refreshed: [List files]

7.3 Restoring S-LEARN-06 (Hot Zone Forensics)

To satisfy the DP-39 requirement, the user must ensure that any task touching "Hot Zones" (Security, Auth, Billing) explicitly references **S-LEARN-06**. This means adding a line to the **Required Context Load**:

- docs/library/skills/S-LEARN-06.md (Hot Zone Forensics)

---

8. Conclusion

The "lost" components of the user's TASK.md are not merely text; they are the **enforcement mechanisms** of the Stela repository. By removing the explicit Closeout pointers, the quantitative results requirements, and the strict decimal schema, the file was downgraded from a **Binding Contract** to a mere **suggestion**, causing the automated governance (policing workflows) to reject the work.

Restoring the file requires a strict adherence to the **DP-OPS-0039** mandate: enforcing the **Pointer-First** logic, reintegrating the **SoP** ledger connection, and strictly formatting the **Results** to meet the "Formally Robust" standard. Only then will the "Dirty State" exception apply, and the repository gates open to accept the work.

Appendix: Key Invariants Checklist

- [ ] **No Contractions:** Linguistic Precision enforcement.
- [ ] **Decimal Headers:** 3.1, 3.2, 3.3 structure.
- [ ] **Closeout Pointer:** Explicit link to Section 4.

- [ ] **SoP Update:** Shipment record included in PR.
- [ ] **Sanitized Tokens:** No `` placeholders; use ENTER_VALUE.
- [ ] **Triple-Bookkeeping:** File + Index + Ledger updated.

Works cited

1. dump-full-main-4b581538.txt